POLITECNICO
MILANO 1863

Ingegneria del software

Design pattern comportamentali

Strategy

Motivazioni

- Si assuma si voglia creare una classe Composition che gestisca elementi organizzati in righe
- A seconda del contesto si possono usare diversi algoritmi per organizzare gli elementi
- Gli algoritmi hanno efficienza diversa a seconda del caso d'uso

Motivazioni

- **Problema:** gestire un insieme di strategie o algoritmi per compiere una determinata azione
- La strategia può essere cambiata a runtime
- Diversi casi applicativi (ad esempio: sorting)

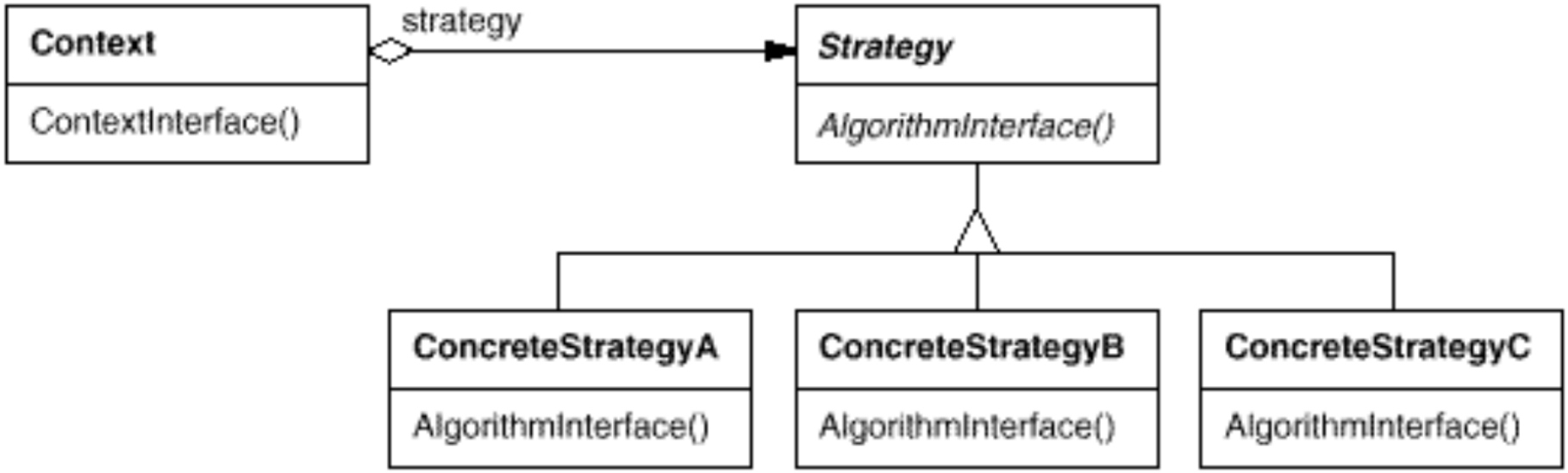
Una soluzione non ideale

```
class Composition {
        private Data data;
        public void traverse() { … }
        public void repair() {
                …
                if (case1) simpleCompose();
                else if (case2) texCompose();
                else arrayCompose();
        }

        public void simpleCompose() { … }
        public void texCompose() { … }
        public void arrayCompose() { … }


}
```

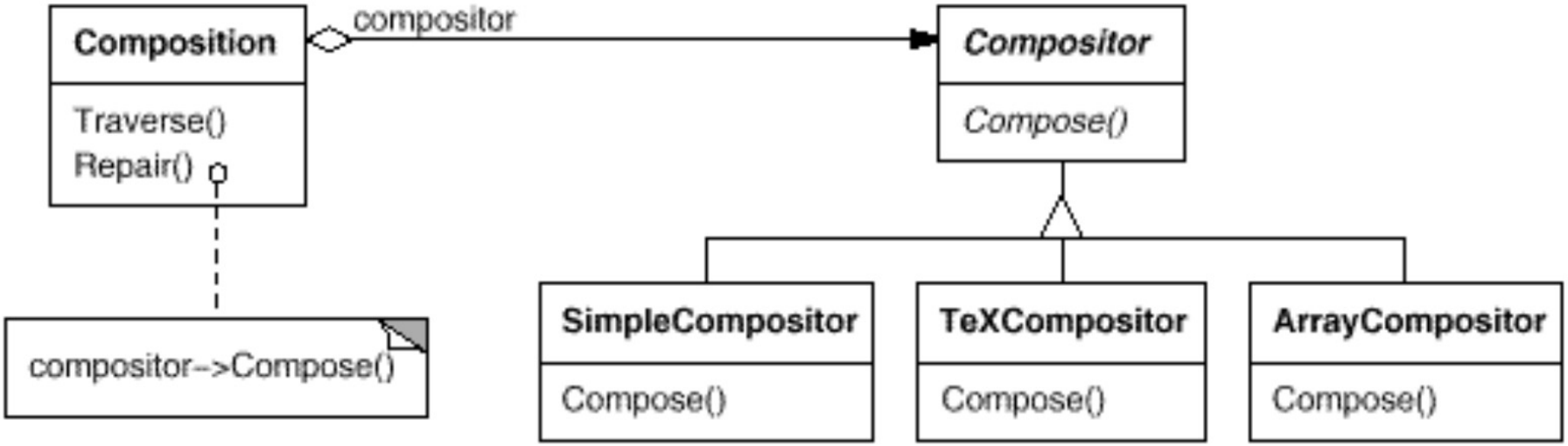
Obbiettivi

- Definire un’interfaccia Strategy
- Ciascuna implementazione di Strategy implementa un diverso algoritmo
- Composition avrà un riferimento ad una Strategy
- La strategia può essere modificata a runtime

Struttura


Context
ContextInterface()
strategy
Strategy
AlgorithmInterface()
ConcreteStrategyA
AlgorithmInterface()
ConcreteStrategyB
AlgorithmInterface()
ConcreteStrategyC
AlgorithmInterface()

Esempio


Composition
Traverse()
Repair()
compositor
Compositor
Compose()
compositor->Compose()
SimpleCompositor
Compose()
TeXCompositor
Compose()
ArrayCompositor
Compose()

Esempio

```
interface Compositor {
      void compose(Data data);
}


class SimpleCompositor implements Compositor {
      public void compose(Data data){
         …
      }
}
```

Esempio

```
class Composition {
        private Data data;
        private Compositor compositor;
        public void setCompositor(Compositor cmp){
                compositor = cmp;
        }
        public void traverse() { … }
        public void repair() {
                compositor.compose(data);
        }
}
```